

Smart Restaurant Ordering System

A Database-Driven Restaurant Ordering and Management Application

Student Shanmukha, Nishanth, Gopal	Roll No. 2520090050,2520090141,2520080032	Course B.Tech — CSIT
Faculty Ms Divya Priya	College KLH University	Academic Year 2026-2027

Problem Statement & Objectives

Problem: Traditional Ordering Challenges

- Incorrect orders; delays; manual billing errors; poor tracking; high staff workload; scattered records

Project Objectives

- Digitize ordering, reduce manual effort, minimize errors, centralize menu & orders, automate billing, maintain organized DB records

Keep the focus on measurable improvements in order accuracy, processing time, and record-keeping.



Proposed System & Key Features

Solution Overview

Centralized application with a relational database powering customer and staff workflows. Role-based interfaces for customers and staff with synchronized order state.

- Real-time order storage and retrieval
- Role-based views (Customer / Staff)
- Automated billing and status updates



Customer Features

- View menu •
- Select items •
- Specify qty •
- Place orders •
- View bill



Staff Features

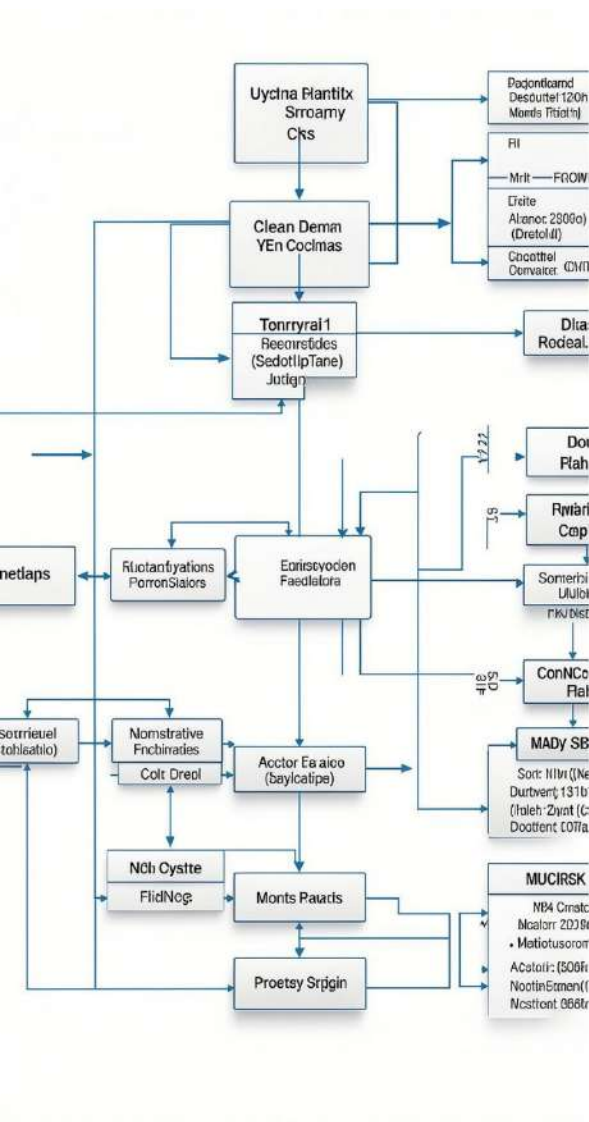
- Manage menu •
- View orders •
- Update status •
- Customer records
- Generate bills

Database Design — Main Tables

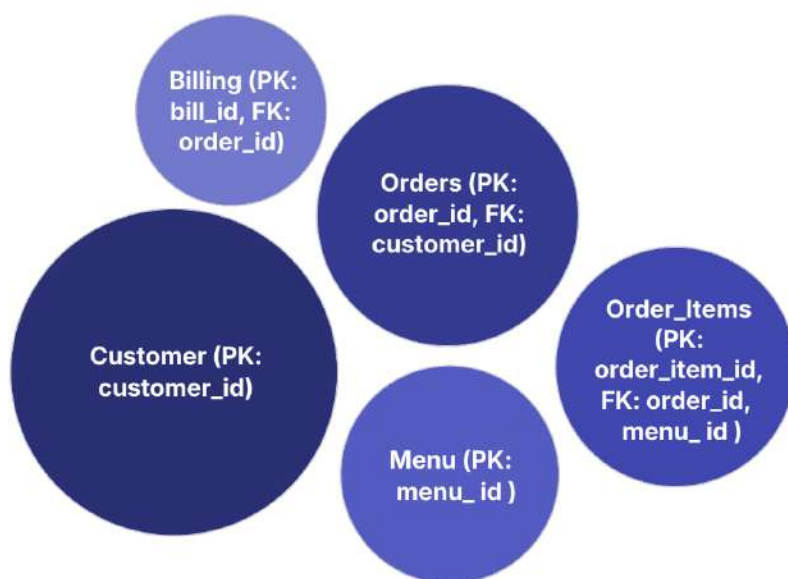
Table	Key Fields (examples)
Customer	customer_id (PK), name, phone, email
Menu	menu_id (PK), item_name, category, price, availability
Orders	order_id (PK), customer_id (FK), order_date, order_status, total_amount
Order_Items	order_item_id (PK), order_id (FK), menu_id (FK), quantity, subtotal
Billing	bill_id (PK), order_id (FK), total_amount, payment_status

Notes

- PK = Primary Key; FK = Foreign Key
- Use indexes on customer_id, menu_id, order_date for performance
- Constraints: NOT NULL, CHECK (price & quantity ≥ 0), UNIQUE (email optional)



ER Diagram



The ER diagram visualizes entities, keys, relationships, and cardinality — the backbone of the relational schema.



SQL & DBMS Concepts

Core Concepts

- Primary Keys • Foreign Keys • Constraints • Transactions • Data Integrity

Common SQL

- `SELECT * FROM Menu;`
- `UPDATE Orders SET order_status = 'Preparing' WHERE order_id = 101;`
- `SELECT c.name, o.order_id, o.order_status FROM Customer c JOIN Orders o ON c.customer_id = o.customer_id;`

Short explanations: SELECT reads menu; UPDATE changes order state; JOIN retrieves combined customer-order info.

Project Demo — Screens & Flow

1. Display Menu

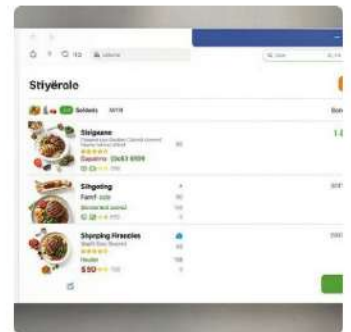
2. Select Food Item

3. Enter Quantity → Place Order

4. Store Order in DB

5. View & Update from Staff Dashboard

6. Generate Bill → Display Records



Results, Advantages & Future Scope

Advantages

- Faster ordering and reduced manual errors
- Automated billing and simple tracking
- Organized records, easy retrieval, and reporting
- Scalable schema for analytics

Future Scope

- Online payments & QR-code ordering
- Customer accounts & notifications
- Table reservations & inventory management
- Sales analytics & mobile app



Conclusion & Thank You

Smart Restaurant Ordering System centralizes customers, menu, orders, order status, and billing using relational DBMS principles — demonstrating DB design, SQL, transactions, and integrity constraints.

Key Takeaway

Reliable, efficient ordering with structured data and clear operational workflows.

Questions?

Ready to demo the application and answer technical questions.

📧 Contact: john.doe@example.edu

